Gaoling School of Artificial Intelligence FUDAN UNIVERSITY

ToolSpace: Scaling Tool-Centric Environment Synthesis for Long-Horizon Agents

Guanting Dong*, **Zhichao Sun***, **Xinyu Yang***, **Rui Wang, Xin Deng, Jiajie Jin, Zhixiong Yang, Junhao Huang, Han Han, Mingyu Ma, Xiaotao Wang, Yuyang Hu, Xiaoshuai Song, Xiaoxi Li, Jiejun Tan, Zhecan James Wang, Shihan Dou, Honglin Guo, Zhiheng Xi, Tao Gui†, Tao Ji, Qi Zhang, Yutao Zhu, Ji-Rong Wen, Zhicheng Dou†**

Renmin University of China Fudan University

*Equal contribution. †Corresponding authors.

Abstract

Large language models are deployed as long-horizon agents that compose diverse tools across web services into executable workflows. Existing web environment-scaling methods are data-centric: they first synthesize a database and scenario, then derive tools and tasks from them. As a result, tool interfaces may not match real ones, tool compositions are not grounded in execution, and tasks rarely span more than one environment. We propose ToolSpace, a tool-centric framework that reverses this order. ToolSpace mines data, state, and retry dependencies from adjacent calls in execution traces of real MCP services and open agent corpora, using deterministic rules rather than LLM judgment, and clusters the resulting weighted tool-composition graph into environments. Databases are synthesized only for state-mutating environments, while query and stateless tools replay recorded returns. ToolSpace then performs state-aware random walks with rollback and decompiles each chain into a natural-language task with assertion-based checks, keeping only samples that replay successfully from the initial state. Applied to AutomationBench and τ^3-Bench Banking, the pipeline produces harder Pro versions on which six frontier models drop by up to 32.9 points. Across seven long-horizon tool-use benchmarks, ToolSpace raises Qwen3.6-35B-A3B from 23.0 to 41.0 average score (43.2 with agent RL) and GLM-5.2 from 44.6 to 56.6, on par with Claude Opus 5. Ablations show that execution-witnessed edges clearly outperform LLM-judged ones, and performance keeps improving with data and environment scale.

Contact: donggaunting@ruc.edu.cn
Project Page: https://toolspace-agent.github.io/
Github: https://github.com/dongguanting/ToolSpace

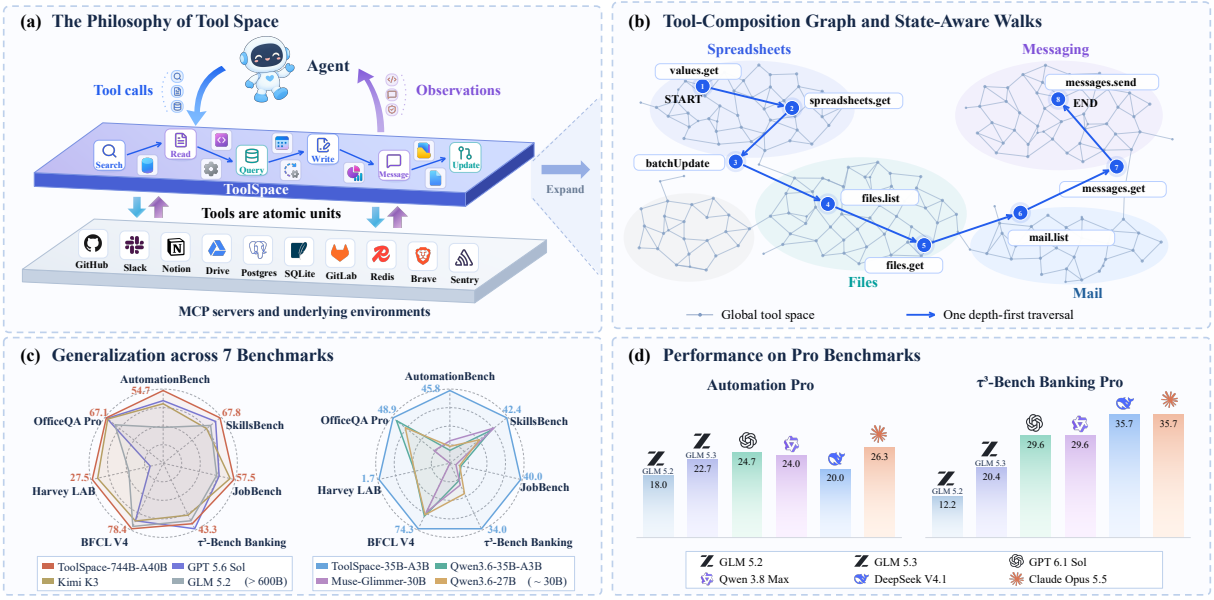


Figure 1: Overview of ToolSpace. (a) Design Philosophy: a tool space layer between agents and MCP environments. (b) The tool-composition graph with a state-aware walk. (c) Generalization across 7 benchmarks. (d) Performance on two Pro benchmarks.

1 Introduction

Recent advances have extended large language models (LLMs) from single-turn chatbots to collaborative agents that complete long-horizon tasks in dynamic environments (Patil et al., 2023; Qin et al., 2023; Schick et al., 2023; Yao et al., 2023). Such agents must not only master a scalable tool set, but also orchestrate diverse tools into executable long-horizon chains. Because tools are the atomic units of agent execution, how tools are organized and composed at scale has become a foundational question for expanding agent capability (Anthropic, 2024; Li et al., 2023; Song et al., 2023; Xu et al., 2025).

To scale tool use, recent work follows two main routes. One family, exemplified by Simia (Li et al., 2025) and GenEnv (Guo et al., 2025), treats the LLM as an implicit textual world model and generates simulated feedback from tool specifications and interaction histories. These methods are inexpensive, but their feedback is prone to hallucination and state consistency is difficult to guarantee. The second family explicitly reproduces interaction logic by synthesizing executable tools together with environment databases: it extracts or constructs an underlying database from a real service theme, generates matching tools, and synthesizes tasks with verifiable outcomes (Dong et al., 2026a; Fang et al., 2025; Song et al., 2026; Wang et al., 2026). Later systems further evolve whole interaction stacks or repair environments from execution feedback (Dong et al., 2026b; He et al., 2026; Mao et al., 2026).

Despite these advances, existing paradigms still follow *Data-Centric Environment Synthesis*: synthesize the underlying database and thematic scenario first, and only then derive tools and tasks. This ordering has three inherent limitations:

- **Tool–functionality mismatch.** Executable tools are synthesized from the environment database and theme description, so their parameter logic easily diverges from real tool interface definitions. Agents consequently learn a synthetic tool protocol rather than operations transferable to a real tool ecosystem.
- **Inauthentic tool orchestration.** Existing work builds tool-link graphs and synthesizes tasks through random linking or shallow LLM judgments, so the resulting links lack authenticity. Real tool services are moreover unstable and noisy, and their logical relations are hard to establish subjectively.
- **Lack of cross-environment task synthesis.** Current task synthesis relies on the data and rules of a single environment, whereas real-world tasks routinely span several environments. Because tools are locked inside their own sandboxes, cross-environment composition has no explicit reachability relation, and long-horizon complexity cannot grow naturally.

Together, these limitations share a single root cause: Data-Centric Environment Synthesis presumes a realistically synthesized environment and derives tool composition from it, reversing the dependency that governs real agent execution. How to synthesize agent training data that preserves real tool orchestration across diverse environments therefore remains an open challenge.

Our starting point is that, ***within agent execution, tools are the atomic units of capability growth, not static environment data***. As illustrated in Figure 1(a)–(b), a *Tool Space* that has not yet been explicitly modeled always lies between an agent and its underlying environment databases, and the relations among tools naturally connect an extensible environment ecosystem. A natural path is therefore to model the real connectivity and state transfer among tools, and to build a cross-environment tool-graph layer on top of these relations. The complexity of synthesized tasks then arises from expanding the real tool-composition space, rather than from the data scale of any single environment.

Following this insight, we propose **ToolSpace**, a tool-centric framework for cross-environment agent data synthesis. Unlike Data-Centric Environment Synthesis, ToolSpace does not mine a new environment ecosystem from scratch; it reconstructs a scalable, explorable tool space from execution traces,

and synthesizes long-horizon agent tasks across the tool and environment clusters connected in this space. ToolSpace consists of two stages:

- **Tool Space Exploration.** We extract tool calls from real-world execution traces and mine data, state, and retry links with deterministic rules, building a scalable tool-composition graph. Tightly coupled tools are clustered into environment clusters; an underlying database is synthesized on demand only for operation environments, while original tool returns are reused directly otherwise. This preserves tool-composition relationships while substantially reducing environment synthesis cost.
- **Cross-Environment Verifiable Task Synthesis.** Over the ToolSpace graph, State-Aware Random Walks with Rollback Sampling sample state-compatible successors by edge weight and execute them step by step, retaining only chains that execute end to end under runtime-observed state transitions. Each retained walk is then decompiled into a natural-language task with assertion-based verification scripts, yielding training samples that can be replayed and audited across environments.

To validate ToolSpace's ability to flexibly synthesize complex cross-environment tasks, we anchor the full pipeline to AutomationBench (Shepard and Salimans, 2026) and τ^3-Bench Banking (Yao et al., 2024), two long-horizon agent benchmarks and extend their difficulty, producing **AutomationBench Pro** and **τ^3-Bench Banking Pro**. The upgraded versions exceed the standard ones in task difficulty and number of environments crossed. Claude Opus 5.5 drops by 32.9 points on AutomationBench Pro relative to the standard version (Figure 1).

In summary, our main contributions are as follows:

- We identify limits of data-centric synthesis: reverse-synthesized tools mismatch real tool semantics, tool graphs lack execution evidence, and tasks are confined to single-environment complexity; and we then propose *Tool-Centric Environment Synthesis*.
- We propose ToolSpace, which reconstructs a scalable, explorable tool-composition space from real execution traces and synthesizes long-horizon, verifiable agent tasks across diverse tool and environment combinations within this space.
- We show that ToolSpace plugs into long-horizon tool-use benchmarks, producing AutomationBench Pro and τ^3-Bench Banking Pro, which exceed their standard versions in difficulty, tool-call turns, and cross-environment breadth.
- Across seven long-horizon tool-use benchmarks, experiments demonstrate clear advantages across model scales (8B–744B). ToolSpace-744B-A40B surpasses GPT 5.6 Sol in average score (56.6 vs. 48.9) and matches Claude Opus 5. Further analyses reveal clear scaling relationships of data scale, environment scale, and task diversity with agent performance.

2 Related Work

Agent Environment Scaling. Agent Environment Scaling has become an important foundation for the evolution of agent capabilities, by scaling the *environments* in which agents act. Simulator-style methods, exemplified by Simia (Li et al., 2025) and GenEnv (Guo et al., 2025), use an LLM as a textual world model to emulate tool feedback: cheap to scale, but prone to hallucination and state inconsistency. Executable-environment methods instead synthesize databases, tools, and verifiable tasks programmatically (Dong et al., 2026a; Fang et al., 2025; Song et al., 2026; Wang et al., 2026), with later variants that evolve environments or whole interaction systems (He et al., 2026; Mao et al., 2026) or repair them from execution feedback (Dong et al., 2026b). In all of these, the unit of scaling remains the environment (or the surrounding harness), so which tools may compose is fixed before any task is drawn. ToolSpace is complementary: it recovers tool-composition structure

witnessed by executions and synthesizes databases only on demand, so the same machinery can *upgrade* benchmarks.

Graph-Based Agentic Data Synthesis. Building agent data with intrinsic dependencies on structured graphs is a promising direction. For search and knowledge agents, prior work builds knowledge graphs or formalizes information seeking tasks to generate hard questions and trajectories (Tao et al., 2025; Wu et al., 2025). For general tool use, function-signature or API-dependency graphs are walked to produce multi-turn calls and verified blueprints (Dong et al., 2026a; Prabhakar et al., 2025; Yin et al., 2025); similar recipes reverse-synthesize trajectories for multimodal GUI and web agents (Zhang et al., 2025). These chains then feed SFT, agentic RL, and reflection-style training (Dong et al., 2026b; Shao et al., 2024; Shinn et al., 2023; Yao et al., 2023). ToolSpace differs in where the graph comes from and what it guarantees: edges are mined by deterministic rules from adjacent real calls (Qin et al., 2023; Wang et al., 2025; Xu et al., 2025; Yang et al., 2024) across environments, typed as data, state, or retry and weighted by witnessed frequency; a walk enters training only after replay from s_0 passes state, trace, and output checks. ToolSpace is the framework to make witnessed tool composition the single primitive behind cross-environment tasks and benchmarks.

Preliminaries and Problem Formulation

We formalize tool-use tasks, data-centric environment synthesis, and the tool-centric synthesis problem addressed by ToolSpace.

Tool-use tasks. Let $\mathcal{F}$ denote a tool space, where each tool $f \in \mathcal{F}$ has input and output schemas α_f and β_f. An environment $e = (\mathcal{D}, \mathcal{F}_e)$ pairs a persistent database $\mathcal{D}$ with a tool subset $\mathcal{F}_e \subseteq \mathcal{F}$ and induces a state space $\mathcal{S}_e$ with snapshots s_t. Given a natural-language task q, an agent policy π_θ produces

$$\tau = (a_1, o_1, \ldots, a_L, o_L), \qquad a_t = \pi_\theta(q, h_t), \quad o_t = f_t(x_t; s_{t-1}), \tag{1}$$

where h_t is the interaction history, f_t is the tool invoked with arguments x_t, o_t is its observation, L is the number of tool calls, and s_t is the environment state after step t. A verifier V maps (q, τ, s_L) to $\{0, 1\}$, or to a graded assertion score; the task is solved iff $V = 1$.

Data-centric environment synthesis. Prior work samples a theme m, synthesizes $\mathcal{D} \sim p(\mathcal{D} \mid m)$, derives tools $\mathcal{F}_e \sim p(\mathcal{F}_e \mid \mathcal{D}, m)$ and tasks $q \sim p(q \mid \mathcal{D}, \mathcal{F}_e)$. Because $\mathcal{F}_e$ is conditioned on a fabricated $\mathcal{D}$, its argument logic need not coincide with any real interface, and cross-environment reachability is undefined: tools from different $(\mathcal{D}, \mathcal{F}_e)$ pairs share no typed relation.

Tool-centric environment synthesis (our problem). Given real execution traces $\mathcal{M} = \{(f_t, x_t, o_t)\}$, we seek: (i) a typed tool graph $\mathcal{G}_{\text{TS}} = (\mathcal{V}, \mathcal{E}, w)$ whose edges are *witnessed* by $\mathcal{M}$; (ii) executable environments recovered or synthesized cluster-wise; and (iii) a dataset $\mathcal{X}_{\text{TS}} = \{(q, V, \omega, s_0)\}$. Each tool chain $\omega = (v_1, \ldots, v_L)$ must be a walk on $\mathcal{G}_{\text{TS}}$ satisfying dependency reachability, state preconditions with respect to s_t, and parameter provenance: every argument must be traceable to q, s_0, or an earlier observation $o_{<t}$. It must also be *replay-verified*, such that re-executing ω from s_0 passes all assertions in V. Our objective is to maximize the long-horizon, cross-environment coverage of $\mathcal{X}_{\text{TS}}$ subject to replay verification, rather than the fidelity of any single synthesized database.

3 Method

Overview. We present **ToolSpace**, a tool-centric framework for cross-environment agent data synthesis (Figure 2). ToolSpace comprises three tightly coupled components, detailed below:

- **Tool Space Exploration** (§3.1) extracts tool calls from real traces, mines data/state/retry dependencies by rules, builds a tool-composition graph, clusters tools into environments, and synthesizes a database only for operation-bearing clusters.

- **Cross-Environment Task Synthesis** (§3.2) explores executable long-horizon chains by State-Aware Random Walks and decompiles them into tasks with verification scripts, retaining only samples that replay from s_0 and pass all checks.
- **AutomationBench Pro and τ^3-Bench Banking Pro** (§4) seamlessly embed ToolSpace chains into the environments of two frontier benchmarks and automatically expand their difficulty while keeping the evaluation protocol aligned.

3.1 Tool Space Exploration

3.1.1 Agent Trajectory Collection from Web

To obtain diverse, large-scale real execution evidence, we build a unified agent-trajectory corpus $\mathcal{M}$ from two source families.

MCP-ecosystem trajectories. Existing web environment scaling work (Dong et al., 2026a; Xu et al., 2025) typically synthesizes task queries over real MCP servers from registries such as Smithery. Because their tool definitions come from real servers, such tasks can be seamlessly re-executed against the live services. We collect a real tool catalog from the registry, keep only tasks whose tool definitions exactly match the real servers, execute them with diverse agents on real MCP services, and record the complete call chains as $\mathcal{M}_{\text{MCP}}$.

Web-sourced agent trajectories. We additionally use a deep-search agent to collect general tool-use trajectories from the web. We group them by execution provenance and apply stricter filtering to less directly grounded data:

- ***Real-server interaction trajectories.*** We retain records executed on real services. Toucan-1.5M (Xu et al., 2025) covers 495 MCP servers, 2,000+ tools, and 1.5M trajectories, with synthetic instructions but real executions. ToolBench/ToolLLM (Qin et al., 2023) covers 16,464 RapidAPI interfaces and ~469K API calls; we remove unavailable interfaces.
- ***Real-workspace interaction trajectories.*** We collect trajectories executed by SWE-agent (Yang et al., 2024) and OpenHands (Wang et al., 2025) in real containers and repositories. We retain only explicit tool calls and exclude shell wrappers such as `execute_bash`.
- ***Synthetic-tool trajectories.*** We collect trajectories generated in synthetic environments (Dong et al., 2026a,b; Song et al., 2026) and retain calls whose tool names and schema formats match corresponding real tools.

The three groups merge into $\mathcal{M}_{\text{open}}$; all tool calls form

$$\mathcal{M} = \mathcal{M}_{\text{MCP}} \cup \mathcal{M}_{\text{open}} = \{(f_t, x_t, o_t)\}, \tag{2}$$

where (f_t, x_t, o_t) is the t-th call of a trajectory: f_t the tool, x_t its inputs, and o_t its result. Calls within a trajectory keep their order.

3.1.2 Tool-Space Graph Modeling

We organize $\mathcal{M}$ into a typed tool-composition graph $\mathcal{G}_{\text{TS}} = (\mathcal{V}, \mathcal{E}, w)$ that can chain different services into one tool world (Figure 2).

Nodes. For every tool call f occurring in $\mathcal{M}$, we create a node $v_{f,\tilde{x}} = (n_f, \alpha_f, \beta_f, \tilde{x})$, where n_f is the tool name, α_f and β_f are its input and output schemas, and $\tilde{x}$ is its normalized argument configuration. Thus, calls to the same canonical tool with different argument configurations form distinct nodes, as they may produce different outputs and downstream dependencies.

Relations. For adjacent calls (f_t, x_t, o_t) and $(f_{t+1}, x_{t+1}, o_{t+1})$ in a trajectory, we extract three edge types by deterministic rules, *no LLM judgment is used*:

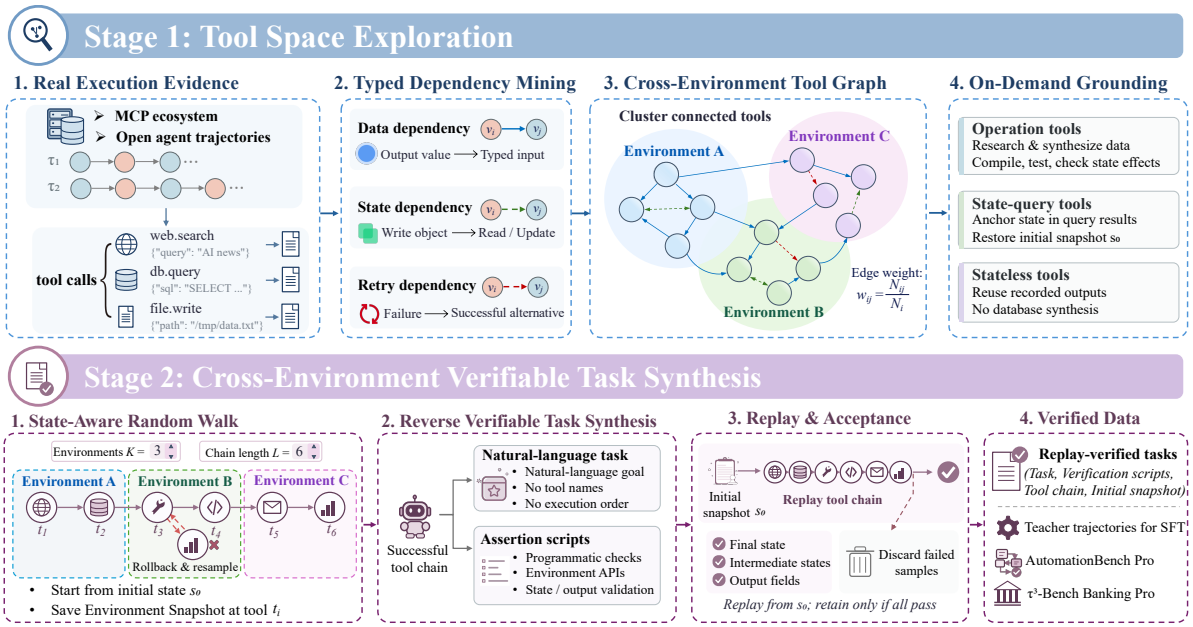


Figure 2: Overview of the ToolSpace framework. Stage 1 builds the Tool Space from real traces; Stage 2 synthesizes cross-environment tasks by State-Aware Random Walks, retaining only samples that pass replay verification.

- ***Data dependency*** ($v_{f_t} \xrightarrow{\text{data}} v_{f_{t+1}}$): the input x_{t+1} uses a value from o_t and field types match. Concrete data is passed.
- ***State dependency*** ($v_{f_t} \xrightarrow{\text{state}} v_{f_{t+1}}$): the first call mutates an environment object and the second call reads or further mutates *the same object*, not merely a coincidentally equal value.
- ***Retry dependency*** ($v_{f_t} \xrightarrow{\text{retry}} v_{f_{t+1}}$): the first call returns an error, the trajectory switches tools, and the next call succeeds. This records failure-driven substitution, not sequencing.

Hence, the edge set is defined as $\mathcal{E} = \mathcal{E}_{\text{data}} \cup \mathcal{E}_{\text{state}} \cup \mathcal{E}_{\text{retry}}$.

Weights. To retain recurring tool dependencies rather than tools that merely appear consecutively, we weight each candidate edge by its observed transition frequency. Let N_i denote the number of occurrences of f_i in $\mathcal{M}$ and N_{ij} the number of times f_j immediately follows f_i. We define

$$w(v_i, v_j) = \frac{N_{ij}}{N_i}.$$

We remove low-frequency edges. For candidate data edges, we additionally require the downstream arguments to be traceable to the upstream output, which prevents false links caused by common constants such as `limit=10`. Consequently, $\mathcal{G}_{\text{TS}}$ captures tool transitions during execution rather than merely listing available tools.

3.1.3 Agent Environment Data Synthesis

ToolSpace materializes underlying data only when an environment depends on persistent state, avoiding the need to synthesize a complete database in advance. We cluster the weighted undirected projection of $\mathcal{G}_{\text{TS}}$ by connection density and organize the clusters using the environment and tool configurations of real MCP services. This produces $\mathcal{C} = \{c_1, \ldots, c_K\}$, where each cluster defines an environment $e_k = (\mathcal{D}_k, \mathcal{F}_k)$ with tool set $\mathcal{F}_k$ and database $\mathcal{D}_k$. We consider three cases:

- ***Operation environments.*** These environments contain tools that create, modify, or delete objects and therefore require an underlying database. Following Agent-World (Dong et al., 2026a), we

derive a theme m_k from each cluster and use a research agent $\mathcal{G}_{\text{res}}$, built on policy model π_θ and tools $\mathcal{T}_{\text{res}}$, to collect and align web data: $\mathcal{D}_k^{(0)} = \mathcal{G}_{\text{res}}(m_k; \pi_\theta, \mathcal{T}_{\text{res}})$. We then iteratively expand it as $\mathcal{D}_k^{(n+1)} = \phi(\mathcal{D}_k^{(n)}, m_k, \mathcal{T}_{\text{res}})$ for $n = 0, \ldots, N-1$. The environment is retained after unit tests and state-transition checks.

- ***State-query environments.*** Many state-query tools return partial snapshots of the environment, such as `google_drive.files.list` and `gmail.users.messages.get`. We use these outputs as anchors to initialize the relevant environment state rather than synthesizing a database from scratch. The resulting state serves as the initial snapshot s_0.
- ***State-free environments.*** These environments contain tools that require no environment-specific persistent state, such as web search and fixed-index lookup. For matching replay calls, we reuse cached outputs from $\mathcal{M}$ without issuing live API calls or synthesizing a database.

The first two types enter the executable environment system $\mathcal{E}_{\text{TS}} = \{e_k\}$, while state-free environments use cached outputs. This design reduces environment-synthesis and API costs and accelerates subsequent trajectory synthesis.

Algorithm 1: ToolSpace

Input: trace corpus $\mathcal{M}$, target length L, target environments K, walk budget
Output: verified dataset $\mathcal{X}_{\text{TS}}$

Build $\mathcal{G}_{\text{TS}} = (\mathcal{V}, \mathcal{E}, w)$ from adjacent calls in $\mathcal{M}$ (data/state/retry rules); prune untraceable or low-weight edges
Cluster $\mathcal{G}_{\text{TS}}$ into environments $\{e_k = (\mathcal{D}_k, \mathcal{F}_k)\}$; synthesize $\mathcal{D}_k$ on demand (operation), restore s_0 (state-query), or cache returns (stateless)
foreach *walk attempt* **do**
    Start at a node bound to $(\mathcal{D}_k, s_0)$; repeatedly sample an admissible neighbor by Eq. (3), execute it, and update s_t
    if *execution fails or violates the traversed edge relation* **then** restore s_{t-1} and resample (rollback)
    if *length L and K environments are reached* **then** record ω with $s_0, \ldots, s_L$
end
foreach $\omega \in \Omega$ **do**
    Decompile ω into task q (no tool names/order) and assertions $V = V_{\text{state}} \cup V_{\text{trace}} \cup V_{\text{out}}$
    Replay ω from s_0; **if** *all assertions pass* **then** add (q, V, ω, s_0) to $\mathcal{X}_{\text{TS}}$
end
Deduplicate $\mathcal{X}_{\text{TS}}$; generate teacher trajectories for retained q and keep those passing V

3.2 Cross-Environment Task Synthesis

3.2.1 State-Aware Random Walks

To collect long-horizon chains that preserve real logical connections, we perform State-Aware Random Walks on $\mathcal{G}_{\text{TS}}$ (Stage 2 of Figure 2). Given a number of crossed environments K and chain length L, a walk starts at a tool node bound to a database $\mathcal{D}_k$ and a snapshot s_0; s_t denotes the t step environment-state snapshot during execution.

Graph Walk. Let v_t be the current node and consider its neighbors in $\mathcal{E}$. A neighbor is kept only if its next-step inputs can be filled from earlier returns or the current snapshot s_t (dependency reachability, state preconditions, parameter provenance). Among the kept neighbors we sample proportionally to edge weight,

$$P(v_{t+1} = v \mid v_t) = \frac{w(v_t, v)}{\sum_u w(v_t, u)}, \tag{3}$$

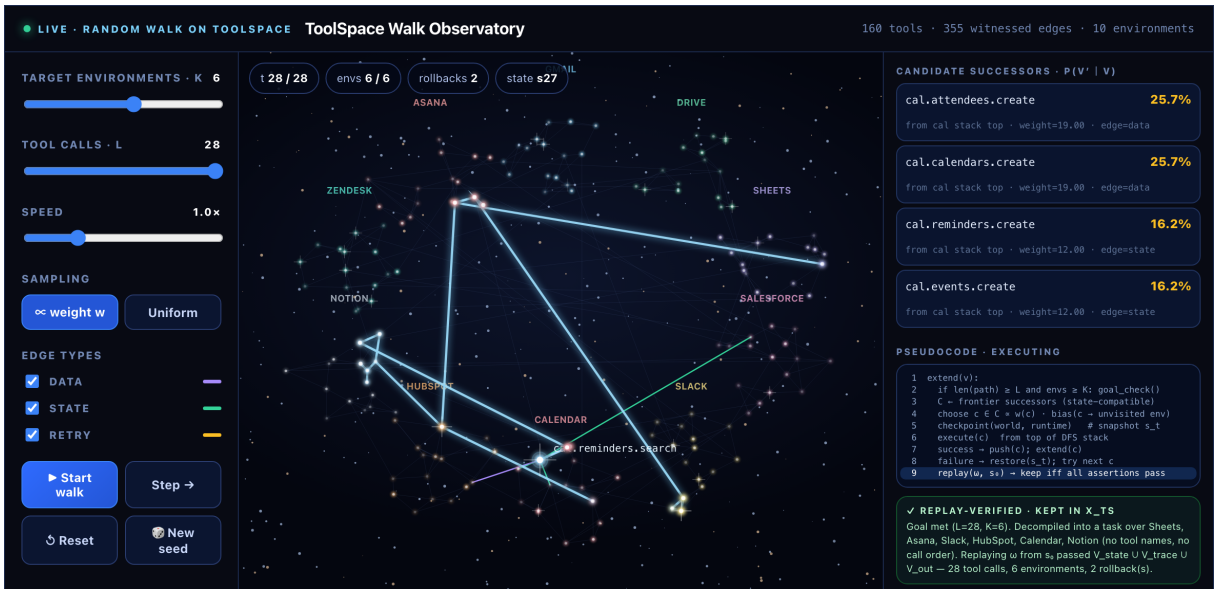


Figure 3: State-aware random walk in action. An example walk from the interactive demo: 28 tool calls across 6 environments, completed after 2 rollbacks and kept after replay verification. Candidate successors and their sampling probabilities $P(v' \mid v)$ are shown on the right.

where the sum ranges only over admissible neighbors. Connections that occur more frequently in real traces are therefore more likely to be extended. The walk continues until its length reaches L and its tools cover K environments, yielding a chain $\omega = (v_1, \ldots, v_L)$.

Rollback Sampling. If a selected tool fails, or the state after execution is inconsistent with the data/state relation of the traversed edge, the step is discarded, the environment is restored to s_{t-1}, and another neighbor of v_{t-1} is resampled. Chains that still cannot proceed after repeated rollbacks are discarded. The retained walk set is $\Omega = \{\omega\}$, each walk carrying s_0 and snapshots $s_1, \ldots, s_L$. Finally, each retained chain is replayed from s_0 (Replay & Acceptance in Figure 2): re-executing its tools against the recorded snapshots guarantees state continuity along the chain and that every tool remains executable. Figure 3 shows one completed walk produced by this procedure in the interactive demo on our project page (https://toolspace-agent.github.io/#demo).

3.2.2 Reverse Verifiable Task Synthesis

To turn executable chains into auditable tasks, we reverse-synthesize each $\omega \in \Omega$. A language model writes a natural-language task q from the chain and its state changes; q contains no tool names and no call order. Instead of relying on a reference answer, we generate a verification set for each chain,

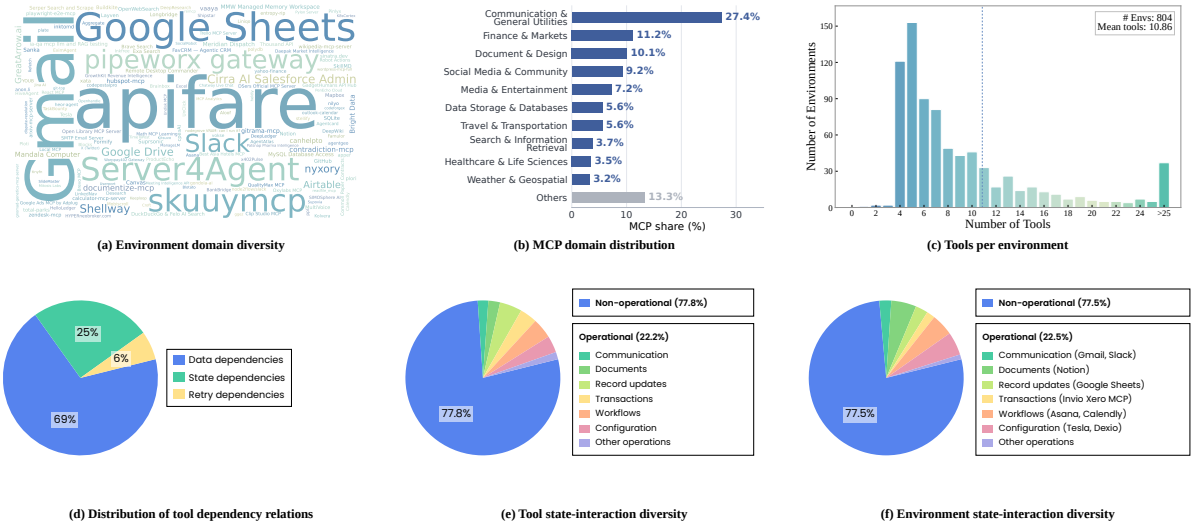
$$V = V_{\text{state}} \cup V_{\text{trace}} \cup V_{\text{out}}, \tag{4}$$

where V_{state} checks the final state s_L, V_{trace} checks key intermediate snapshots, and V_{out} checks fields required to appear in returned results (here V denotes verification, distinct from the node set $\mathcal{V}$). A sample is retained only if re-executing ω from s_0 passes *all* checks in V. Difficulty is raised via L, target K, and the share of cross-environment edges. The dataset is $\mathcal{X}_{\text{TS}} = \{(q, V, \omega, s_0)\}$: task, verification script, tool chain, and snapshot. We present the complete ToolSpace algorithm flowchart in Algorithm 1.

3.3 Detailed Statistics of ToolSpace

Figures 4 and 5 characterize ToolSpace's graph, environments, and synthesized tasks.

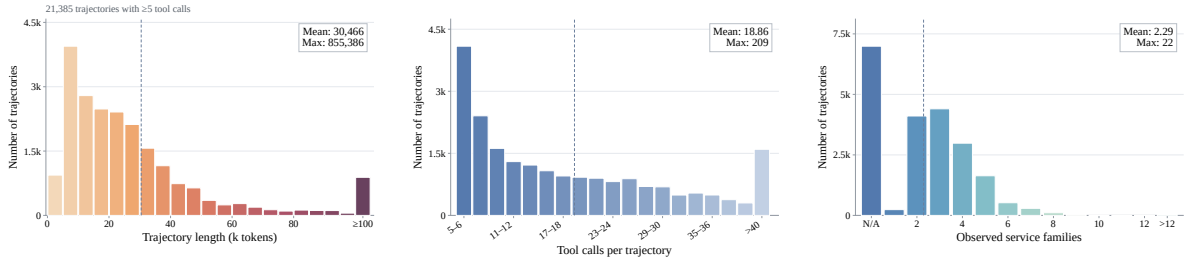
(A) Graph Scale and Dependency Structure. Reconstructed from collected tool-call trajectories, $\mathcal{G}_{\text{TS}}$ contains 77,760 tool-call-pattern nodes derived from 2,042 distinct canonical tools and 178,848



(a) Environment domain diversity (b) MCP domain distribution (c) Tools per environment

(d) Distribution of tool dependency relations (e) Tool state-interaction diversity (f) Environment state-interaction diversity

Figure 4: ToolSpace graph statistics. (a) Environment-name word cloud. (b) Environment distribution across MCP domains. (c) Number of tools per environment. (d) Proportions of data, state, and retry dependencies. (e) Tool distribution by operational category. (f) Environment distribution by operational category.



(a) Trajectory token length (b) Tool calls per trajectory (c) Observed service breadth

Figure 5: Synthesized task and trajectory statistics. Distributions across 21,385 trajectories with at least five tool calls: (a) trajectory token length, (b) tool calls per trajectory, and (c) service families per trajectory.

execution-witnessed dependency edges. Its mean and maximum outdegrees are 2.3 and 56, respectively. Figure 4(a) visualizes real MCP services such as Gmail and Brave Search. Panel (d) shows that data, state, and retry edges account for 69%, 25%, and 6%, respectively. The dominance of data edges indicates that tool composition is driven primarily by intermediate-result transfer, while state and retry edges capture state prerequisites and failure recovery. Every edge is witnessed by adjacent calls in real trajectories, providing execution-grounded tool reachability.

(B) Environment Diversity and Selective Grounding. The reference inventory covers 804 real MCP environments. Figure 4(b) identifies Communication & General Utilities, Finance & Markets, and Document & Design as the largest domains, accounting for 27.4%, 11.2%, and 10.1% of environments, respectively. Panel (c) reports an average of 10.86 matched tools per environment and a maximum of 436; tools such as web search, file reading, and email sending are available across multiple environments. Panel (e) distinguishes 453 operational tools (22.2%) from 1,589 non-operational tools (77.8%), with structured record mutation being the largest operational category at 95 tools. Among the 803 environments with assigned tools, Panel (f) identifies 181 operational (22.5%) and 622 non-operational environments (77.5%), with document persistence being the largest operational category at 42 environments. An environment is operational if it contains at least one tool capable of modifying its state. ToolSpace materializes databases on demand for operational workspaces, restores snap-

shots for state-query tools, and reuses cached returns for stateless tools, reducing database-synthesis requirements and costs relative to data-centric methods.

(C) Task Scale and Complexity. The SFT dataset contains 34,376 trajectories. Figure 5 summarizes the 21,385 trajectories with at least five tool calls. Panel (a) reports mean and maximum lengths of 30,466 and 855,386 tokens, while Panel (b) reports an average of 18.86 tool calls per trajectory and a maximum of 209, indicating sustained multi-step orchestration. Panel (c) shows an average of 2.29 service families per trajectory and a maximum of 22, with at least 66.2% of trajectories involving two or more families. During synthesis, tool chains are sampled through state-aware random walks, executed step by step, and retained only after replay verification. The resulting tasks therefore require both long-horizon tool orchestration and consistent environment state.

4 AutomationBench Pro and τ^3-Bench Banking Pro

To test whether ToolSpace can raise the cross-environment and long-horizon difficulty of frontier benchmarks, we anchor the pipeline to AutomationBench (Shepard and Salimans, 2026) and τ^3-Bench Banking (Shi et al., 2026), showing the pipeline's flexible difficulty expansion.

Benchmark Construction. For a benchmark b with original environment $e_b^{\text{orig}} = (\mathcal{D}_b^{\text{orig}}, \mathcal{F}_b^{\text{orig}})$, we collect its real tool calls $\mathcal{M}_b$, build a tool graph $\mathcal{G}_b$ by the rules of §3.1.2, and thereby link in tools outside the original benchmark that share data or state relations with benchmark tools, expanding a single environment into a cross-environment cluster. We then restore or synthesize the necessary databases and snapshots (§3.1), walk with larger L and K than the original benchmark, and reverse-generate tasks and verification scripts. After replay and deduplication we obtain AutomationBench Pro (600 tasks) and τ^3-Bench Banking Pro (98 tasks). To ensure that Pro benchmarks reflect genuine difficulty rather than task quality, three doctoral-level experts independently reviewed all candidate tasks and their verification scripts over a two-day period. Only tasks approved by at least two experts were retained. Inter-rater agreement was strong, with Fleiss' $\kappa = 0.82$ and an observed agreement of 93.1%.

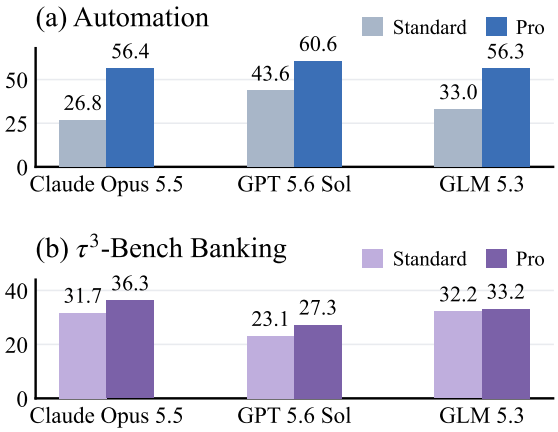


Figure 6: Average tool calls of frontier models on (a) Automation and (b) τ^3-Bench Banking (Standard vs. Pro).

Data Statistics. Figure 6 shows that Pro tasks require substantially more tool calls than the standard splits on both benchmarks. On AutomationBench, average tool calls rise sharply for every model, from 26.8–43.6 under Standard to 56.3–60.6 under Pro. On τ^3-Bench Banking, average calls likewise rise for every model, from 23.1–32.2 to 27.3–36.3, confirming the added difficulty comes from longer chains, not a changed protocol.

5 Experiments

5.1 Experimental Setup

Benchmarks. We evaluate on seven long-horizon tool-use benchmarks, referred to by their full names throughout: **AutomationBench** (Shepard and Salimans, 2026) (workflow automation), **SkillsBench** (Li et al., 2026a) (skill composition), **JobBench** (Li et al., 2026b) (job-style professional tasks), **τ^3-Bench Banking** (Shi et al., 2026) (banking tool–dialogue interaction), the **Berkeley Function Calling Leaderboard V4 (BFCL V4)** (Patil et al., 2025) (function calling), **Harvey LAB** (Harvey AI, 2026) (legal agent workflows), and **OfficeQA Pro** (Opsahl-Ong et al., 2026) (office document question answering).

Baselines. We evaluate four model groups: (i) *proprietary frontier models*: Claude Opus 5 (Anthropic, 2026b) and GPT 5.6 Sol (OpenAI, 2026a); (ii) *open-weight models above 200B parameters*: GLM 5.3 (Zhipu AI, 2026b), GLM 5.2 (Zhipu AI, 2026a), DeepSeek-V4-Pro-0813 (DeepSeek-AI et al., 2026), Kimi K3 (Moonshot AI, 2026), and Qwen 3.8 Max (Qwen Team, 2026c); (iii) *7–8B environment-scaling models*: EnvScaler-8B (Song et al., 2026), AWM-8B (Wang et al., 2026), Simulator-8B (Li et al., 2025), Agent-World-8B (Dong et al., 2026a), Agent-Reflex-8B (Dong et al., 2026b), and their Qwen3-8B backbone (Yang et al., 2025); and (iv) *27–35B open-weight models*: Qwen3.6-27B (Qwen Team, 2026a), Qwen3.6-35B-A3B (Qwen Team, 2026b), Nex-N2 Mini (Nex-AGI, 2026), Muse-Glimmer-30B (Meta, 2026), and WorkRover-35B-A3B (He et al., 2026). For the Pro benchmark, we further evaluate Claude Opus 5.5 (Anthropic, 2026a), GPT 6.1 Sol (OpenAI, 2026b), and DeepSeek-V4.1-Flash (DeepSeek-AI, 2026).

Implementation Details. We train long-horizon agents on three backbones, Qwen3-8B, Qwen3.6-35B-A3B, and GLM-5.2-744B-A40B (the resulting 744B model is denoted ToolSpace-744B-A40B). Detailed ToolSpace statistics are reported in §3.3.

Trajectory collection and environment synthesis. We collect over 200,000 agent–tool interaction trajectories as raw material for ToolSpace construction, with strict quality control during collection. We exclude any trajectory overlapping with an evaluation test set, ensuring that the synthesized data are free of test-set contamination. All data sources and MCP services used are covered by open-source licenses. We mine executable environments following the agentic exploration procedure and settings of Agent-World Dong et al. (2026a).

Task and trajectory synthesis. We use K3 for both teacher-trajectory generation and reverse verifiable task synthesis. All synthesized samples are validated against V using verification scripts, yielding 34,376 SFT samples and 653 RL samples from the same ToolSpace synthesis pipeline, except that RL candidates undergo additional Pass@16 screening with the 35B model, retaining only candidates with one or two correct rollouts to ensure informative RL training.

Training and evaluation. All data-synthesis baselines use the same SFT configuration: the 8B and 35B models are trained for one epoch on 16 NVIDIA GPUs, while the 744B model is trained for one epoch on 256 GPUs. For agent RL, we use standard GRPO Shao et al. (2024) for 48 training steps, with a learning rate of 2×10^{-6}, a total training batch size of 256, and a rollout group size of 16. Following Agent-World, rewards lie in $[0, 1]$, with a reward of 1 assigned only to fully correct trajectories. Inference uses the official recommended settings, each benchmark follows its standard evaluation protocol, and all ToolSpace results are averaged over three independent runs.

Evaluation settings and result provenance. We prioritize official leaderboard results, followed by provider-reported scores under the official protocol and, when unavailable for open-weight models, local reproduction using the official harness, split, metric, tool-call protocol, and recommended inference settings. Local results are averaged over three independent runs; external scores are used as published. For each model–benchmark pair, we record the benchmark version, split, model snapshot, harness, reasoning effort, tool budget, source, and access date. We will release all code, the ToolSpace graph, the Pro benchmarks, and the corresponding model checkpoints.

5.2 Main Results

Long-horizon tool use and office productivity. Table 1 reports performance on seven benchmarks. ToolSpace is strong across model scales; three observations summarize this.

(1) Environment scaling learns the form of function calling, not tool orchestration. All 7–8B environment scaling baselines remain weak (<10) on the cross-environment, long-horizon suites AutomationBench, τ^3-Bench Banking, JobBench, and Harvey LAB. Single-environment data therefore correct the *form* of calls without teaching cross-environment composition.

(2) ToolSpace exhibits cross-environment long-horizon execution across scales. Gains hold in each domain: ToolSpace-8B leads same-size environment scaling baselines on real productivity suites such as AutomationBench, JobBench, and OfficeQA Pro; ToolSpace-35B-A3B exceeds its Qwen backbone on all seven benchmarks, with AutomationBench rising by more than 35 points (8.2→45.8). The pattern persists to 744B: ToolSpace-744B-A40B surpasses GLM 5.3, GPT 5.6 Sol, and its teacher policy K3 on average, and matches Claude Opus 5. These consistent gains across model scales, including performance beyond the teacher, argue against simple capability distillation and instead indicate that ToolSpace teaches transferable tool-composition structure for cross-environment execution.

(3) On-policy agent RL further unlocks capacity. SFT alone lifts ToolSpace by 18.0 points over the 35B backbone (23.0→41.0). On-policy RL on ToolSpace data adds a further 2.2 points at 35B (41.0→43.2), with the largest gain on τ^3-Bench Banking (34.0→42.3), showing headroom under environment RL.

Table 1: Performance on seven tool-use benchmarks. Bold/underline mark the best/second-best score per column *within each same-scale pool* (8B: environment scaling baselines; 27–35B: open-weight models; 744B: >200B and proprietary models).

Model	Size	Automation Bench	Skills Bench	Job Bench	τ^3-Bench Banking	BFCL V4	Harvey LAB	OfficeQA Pro	Avg.
Proprietary Models									
Claude Opus 5	–	45.6	63.7	**68.0**	48.7	**80.6**	22.7	**67.2**	**56.6**
GPT 5.6 Sol	–	47.1	62.5	45.4	46.9	68.8	5.0	66.4	48.9
Open-Weight Models (>200B total parameters)									
GLM 5.3	744B	51.5	63.3	58.2	40.2	74.1	22.5	65.8	53.7
GLM 5.2	744B	29.3	56.7	43.4	37.1	75.6	13.3	56.9	44.6
DeepSeek-V4-Pro-0813	1.7T	41.7	65.0	54.1	44.3	71.4	6.7	66.2	49.9
Kimi K3	2.8T	44.8	51.9	54.3	37.1	69.1	25.4	65.4	49.7
Qwen 3.8 Max	2.4T	49.7	66.7	52.7	**55.2**	66.8	24.2	66.2	54.5
Environment Scaling Baselines (7–8B)									
Qwen3-8B	8B	0.0	0.1	0.1	4.1	42.8	0.0	3.0	7.2
EnvScaler-8B	8B	0.0	1.6	1.1	7.2	47.5	0.0	1.5	8.4
AWM-8B	8B	0.0	1.4	0.5	5.2	42.0	0.0	5.3	7.8
Simulator-8B	8B	0.0	0.4	0.1	1.0	20.5	0.0	6.0	4.0
Agent-World-8B	8B	0.3	2.5	0.7	6.2	49.0	0.0	3.8	8.9
Agent-Reflex-8B	8B	0.5	3.3	0.8	3.1	51.3	0.0	5.3	9.2
Open-Weight Foundation Models (27–35B)									
Qwen3.6-27B	27B	10.7	22.1	5.6	15.8	58.5	0.8	38.4	21.7
Qwen3.6-35B-A3B	35B	8.2	30.7	6.4	9.3	60.0	0.8	45.9	23.0
Nex-N2 Mini	35B	13.2	16.8	11.5	18.6	63.3	0.0	38.4	23.1
Muse-Glimmer-30B	30B	14.2	32.9	3.5	11.3	55.5	0.0	13.5	18.7
WorkRover-35B-A3B	35B	28.7	–	36.9	–	–	–	48.9	–
ToolSpace (Ours)									
ToolSpace-8B	8B	32.3	3.9	13.1	10.3	65.3	0.0	**12.0**	19.6
+ *Agent RL*	8B	**34.3**	**4.1**	**15.5**	**11.3**	**67.5**	0.0	**12.0**	**20.7**
ToolSpace-35B-A3B	35B	**45.8**	42.4	**40.0**	34.0	74.3	1.7	48.9	41.0
+ *Agent RL*	35B	**45.8**	**45.3**	**40.0**	**42.3**	**74.8**	**2.5**	**51.9**	**43.2**
ToolSpace-744B-A40B	744B	**54.7**	**67.8**	57.5	43.3	78.4	**27.5**	67.1	**56.6**

Table 2: Standard vs. Pro. Std. denotes the standard split; Pro denotes our upgraded split. Δ is Pro−Standard.

Frontier model	AutomationBench			τ^3-Bench Banking		
	Std.	Pro	Δ↓	Std.	Pro	Δ↓
Claude Opus 5.5	59.2	26.3	−32.9	51.5	35.7	−15.8
GPT 6.1 Sol	50.8	24.7	−26.1	46.4	29.6	−16.8
DeepSeek V4.1	50.0	20.0	−30.0	47.4	35.7	−11.7
GLM 5.3	51.5	22.7	−28.8	40.2	20.4	−19.8
Qwen 3.8 Max	49.7	24.0	−25.7	55.2	29.6	−25.6
GLM 5.2	29.3	18.0	−11.3	37.1	12.2	−24.9

5.3 Results on AutomationBench Pro and τ^3-Bench Banking Pro

To test whether ToolSpace raises frontier-benchmark difficulty, Table 2 compares six frontier models on the two Pro splits with Standard. All six frontier models drop substantially from Standard to Pro. Even Claude Opus 5.5 and GPT 6.1 Sol fall by 32.9 and 26.1 on AutomationBench, and by a further 15.8 and 16.8 on τ^3-Bench Banking. Together with Figure 6, where Pro tasks require more tool calls, this shows a sharp rise in cross-environment and tool-execution difficulty, and underscores ToolSpace's flexible difficulty control.

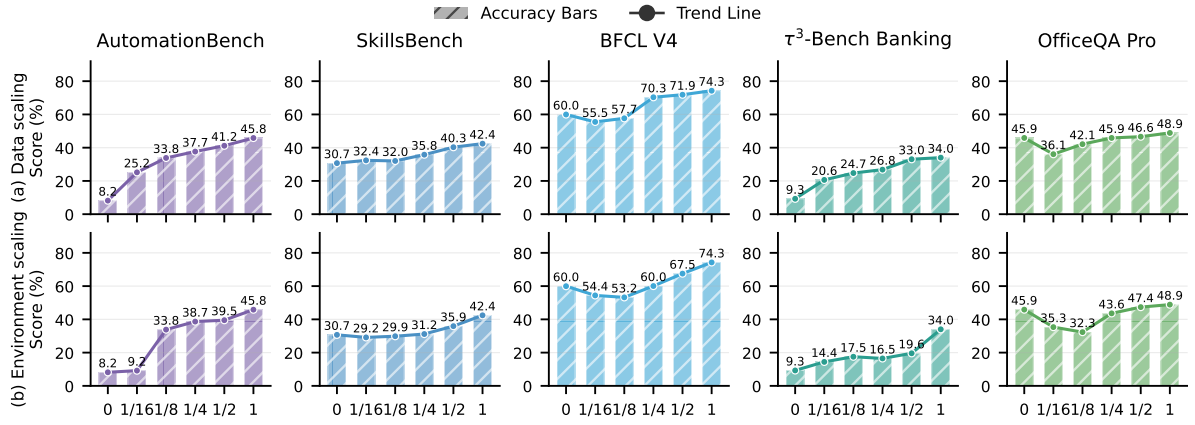


Figure 7: Data and environment scaling results. (a) Data scaling by training-data fraction. (b) Environment scaling by training-environment fraction. Zero denotes the untuned Qwen3.6-35B-A3B backbone.

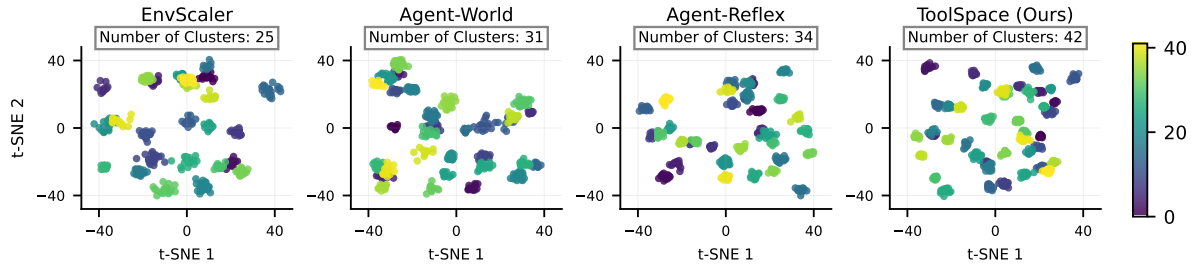


Figure 8: Task-Synthesis Diversity Visualization. Synthesized task embeddings colored by DBSCAN cluster.

5.4 Quantitative Analyses

Environment Scaling Analysis. To examine whether the breadth of the tool space matters independently of data volume, we fine-tune Qwen3.6-35B-A3B on data synthesized from increasing fractions of ToolSpace environments, from 1/16 to the full environment set. The backbone, training configuration, and evaluation protocol are held fixed, and zero denotes the untuned backbone. Figure 7(b) reports the results. Performance improves overall with scale and reaches the full-setting optimum, with intermediate decreases on BFCL V4, OfficeQA Pro, and τ^3-Bench Banking. The increase on AutomationBench between 1/16 and 1/8 is particularly sharp, from 9.2 to 33.8. This suggests that new environment clusters contribute new cross-environment edges, and that space breadth is the limiting factor when coverage is low. In contrast, τ^3-Bench Banking improves more slowly under environment scaling, which is consistent with its reliance on a narrower set of stateful banking tools. This supports the central motivation of ToolSpace: performance scales with the breadth of environments that witnessed chains connect. In marginal terms, AutomationBench gains about +4 points per doubling beyond 1/8 on average, despite a plateau between 1/4 and 1/2, while τ^3-Bench Banking shows its largest jump only at the full environment set (19.6→34.0), indicating that highly connected environment clusters contribute late.

Data Scaling Analysis. To examine whether the benefit of ToolSpace grows with the amount of synthesized data, we fine-tune Qwen3.6-35B-A3B on increasing fractions of the ToolSpace SFT dataset, namely 1/16, 1/8, 1/4, 1/2, and the full dataset. The backbone, training configuration, and evaluation protocol are held fixed, and zero denotes the untuned backbone. Figure 7(a) reports the results. Performance improves overall as more data are used, and the full dataset performs best on all five benchmarks. SkillsBench shows a small intermediate decrease from 32.4 at 1/16 to 32.0 at 1/8.

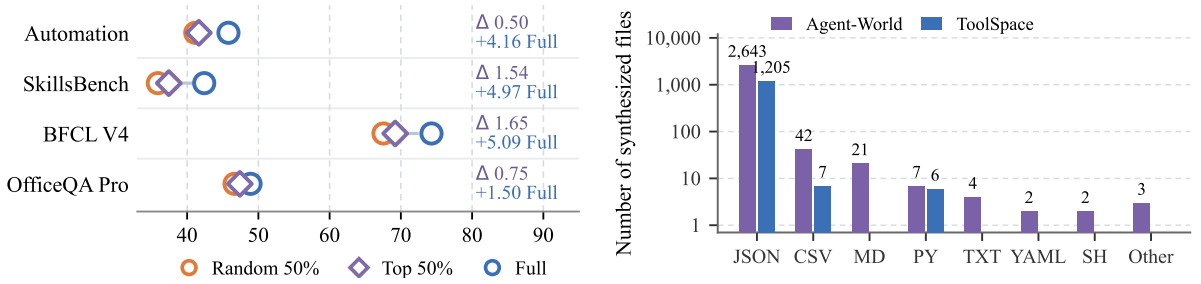


Figure 9: Environment relevance and synthesis cost. (a) Effect of environment relevance: Full, Top 50%, and Random 50% environments; Δ is the Top–Random gap. (b) Environment synthesis cost proxy: synthesized files by type over 500 sampled environments.

AutomationBench rises from 25.2 at 1/16 to 45.8 with the full dataset, and τ^3-Bench Banking rises from 20.6 to 34.0. The gain is therefore not explained by a small set of high-quality examples. Each additional fraction contributes new witnessed tool chains and dependency structures, which is consistent with our motivation that capability grows from the composition space covered by the data. The small decreases below the backbone at the 1/16 fraction on BFCL V4 and OfficeQA Pro indicate interference when the SFT mixture is too small, and the trend becomes stable once the data fraction increases. This demonstrates the scalability of ToolSpace's synthesized data. In marginal terms, the step from 1/2 to the full dataset still improves all five benchmarks (AutomationBench +4.6), with no plateau, indicating that the current synthesis scale has not yet hit the returns ceiling of data volume.

Environment Relevance Analysis. To verify that ToolSpace's gains do not stem from overfitting to benchmark-like environments, we rank environment clusters by relevance to each benchmark using BGE (Chen et al., 2024) embeddings, and compare SFT on (i) the full data, (ii) only the top-50% most relevant environments (Top-50%), and (iii) a random 50% of environments (Random-50%) (Figure 9(a)). If the gains were driven by benchmark-specific environment fitting, Top-50% should substantially outperform Random-50%, which uses the same number of environments but excludes many of the most relevant ones. Instead, the two differ by at most 1.7 points on AutomationBench, SkillsBench, BFCL V4, and OfficeQA Pro, and both approach full-data performance using only half the environments. ToolSpace's gains therefore do not hinge on benchmark-aligned environments, but reflect transferable tool-composition structure.

Task-Synthesis Diversity Visualization. We examine whether ToolSpace spans a broader space of environments and tasks than prior environment-scaling methods by randomly sampling 2k questions from four SFT corpora (EnvScaler, Agent-World, Agent-Reflex, and ToolSpace). We employ BGE-M3 (Chen et al., 2024) as the embedding model, apply t-SNE for dimensionality reduction, and use DBSCAN (Ester et al., 1996) for clustering. As shown in Figure 8, ToolSpace forms the largest number of question clusters (42), compared with EnvScaler, Agent-World, and Agent-Reflex (25, 31, and 34, respectively), while retaining compact local groupings and broader coverage of the embedding space. This pattern suggests that ToolSpace expands task-space coverage beyond prior environment-scaling methods. We attribute the gain to edge-weighted sampling over observed tool compositions, which connects a wider range of environments.

Environment Synthesis Cost Analysis.

We estimate ToolSpace's environment synthesis cost savings using the number of synthesized files as a proxy. From 500 environments randomly sampled from Agent-World's public library, we identify the files that ToolSpace would need to synthesize. Because file generation and validation complexity may vary, this proxy does not directly measure token, wall-clock, API, or compute cost.

As shown in Figure 9(b), Agent-World synthesizes all 2,724 files in the sample, whereas ToolSpace synthesizes only 1,218 files associated with operation-bearing environments—1,506 fewer files, or a 55.3% reduction. Specifically, JSON files decrease from 2,643 to 1,205 and CSV files from 42 to 7.

Table 3: Component ablation. Darker shading marks a larger within-column drop from Full.

Variant	Automation Bench	Skills Bench	BFCL V4	τ^3-Bench Banking	OfficeQA Pro
Full ToolSpace	**45.8**	**42.4**	**74.3**	**34.0**	**48.9**
w/o state edges	36.5	38.0	70.1	21.5	44.0
w/o data edges	38.2	37.5	62.4	29.8	40.6
w/o retry edges	43.9	41.2	72.8	32.6	47.9
w/o rollback	41.0	39.8	71.5	30.1	46.3
Uniform walk (no w)	39.6	38.9	69.8	28.4	45.1
LLM-judged edges	37.8	36.4	66.9	26.7	43.5

This reduction follows from ToolSpace's selective-grounding strategy: persistent artifacts are materialized only for operation-bearing environments, while recorded returns are reused for stateless interactions. The lower artifact-generation workload indicates reduced environment synthesis cost, although exact token, time, and compute savings are not measured.

Component Ablation. Table 3 reports a component ablation of ToolSpace at 35B, where w/o denotes removing or replacing one specific component. We make three observations. **(1)** Removing any component degrades all five benchmarks, showing that the components are complementary rather than redundant. **(2)** The two structural edge types divide the labor: state edges matter most for τ^3-Bench Banking and AutomationBench, where state preconditions decide whether a chain is executable at all, while data edges matter most for BFCL V4 and OfficeQA Pro, where argument provenance carries the task. Their comparable average drops (7.1 vs. 7.4 points) fall on disjoint benchmarks, confirming that state dependencies, not data flow alone, enable cross-environment execution. **(3)** How edges are obtained and traversed matters as much as which edges exist: LLM-judged edges degrade every benchmark almost as much as removing state edges, so edges must be witnessed by execution rather than merely plausible; rollback and edge-weighted sampling contribute in between, and retry edges have the smallest effect, consistent with their role in robustness to noisy tools. Environment relevance is ablated in §5.4 above.

6 Conclusion

In this paper, we present ToolSpace, a tool-centric framework for cross-environment agent data synthesis. By reconstructing a typed, execution-witnessed tool-composition graph from real traces, materializing databases only on demand, and synthesizing tasks by state-aware walks with replay-based assertion verification, ToolSpace turns real tool orchestration, rather than fabricated environments, into the source of long-horizon complexity. It upgrades existing benchmarks (AutomationBench Pro, τ^3-Bench Banking Pro), on which frontier models drop by up to 32.9 points, and trains agents that outperform strong baselines across 8B-744B scales on 7 benchmarks. Further analyses indicate that witnessed state and environment breadth contribute to these gains.

References

Anthropic. Model context protocol (MCP) specification. https://modelcontextprotocol.io, 2024. Accessed 26 Oct. 2026.

Anthropic. System card: Claude opus 5.5. https://www-cdn.anthropic.com/fc1b44717c85dc068bc6ba5024219938094694bd/Claude%20Opus%205.5%20System%20Card.pdf, September 2026a. Released September 22, 2026.

Anthropic. Introducing Claude Opus 5. https://www.anthropic.com/news/claude-opus-5, 2026b. Official announcement, 24 Jul. 2026; accessed 26 Oct. 2026.

Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. M3-embedding: Multilinguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 2318–2335, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.137. https://aclanthology.org/2024.findings-acl.137/.

DeepSeek-AI. Deepseek-v4.1-flash: Pushing the limits of kv cache compression, 2026.

DeepSeek-AI, Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chenchen Ling, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chengyu Hou, Chenhao Xu, Chenze Shao, Chong Ruan, Conner Sun, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Donghao Li, Dongjie Ji, Erhang Li, Fang Wei, Fangyun Lin, Fangzhou Yuan, Feiyu Xia, Fucong Dai, Guangbo Hao, Guanting Chen, Guoai Cao, Guolai Meng, Guowei Li, Han Yu, Han Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoling Zhang, Haoming Luo, Haoran Wei, Haotian Yuan, Haowei Zhang, Haowen Luo, Haoyu Chen, Haozhe Ji, Hengqing Zhang, Honghui Ding, Hongxuan Tang, Huanqi Cao, Huazuo Gao, Hui Qu, Hui Zeng, J Yang, JQ Zhu, Jia Luo, Jia Song, Jia Yu, Jialiang Huang, Jialu Cai, Jian Liang, Jiangting Zhou, Jiasheng Ye, Jiashi Li, Jiaxin Xu, Jiewen Hu, Jieyu Yang, Jin Chen, Jin Yan, Jingchang Chen, Jingli Zhou, Jingting Xiang, Jingyang Yuan, Jingyuan Cheng, Jingzi Zhou, Jinhua Zhu, Jiping Yu, Joseph Sun, Jun Ran, Junguang Jiang, Junjie Qiu, Junlong Li, Junmin Zheng, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Kexing Zhou, Kezhao Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Wang, Leyi Xia, Li Zhang, Liang Zhao, Lihua Guo, Lingxiao Luo, Linwang Ma, Linyan Zhu, Litong Wang, Liyu Cai, Liyue Zhang, Longhao Chen, MS Di, MY Xu, Max Mei, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Mingxu Zhou, Minmin Han, Ning Wang, Panpan Huang, Panpan Wang, Peixin Cong, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Qiwei Jiang, Rui Tian, Ruifan Xu, Ruijie Lu, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqian Chen, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, Ruyi Chen, SH Liu, Shanghao Lu, Shangmian Sun, Shangyan Zhou, Shanhuang Chen, Shaofei Cai, Shaoheng Nie, Shaoqing Wu, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, Shuying Yu, Songyang Zhou, Tao Ni, Tao Yun, Tian Jin, Tian Pei, Tian Ye, Tianle Lin, Tianran Ji, Tianyi Cui, Tianyuan Yue, Tingting Yu, Tun Wang, W Zhang, WL Xiao, Wangding Zeng, Wei An, Weilin Zhao, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjing Yao, Wenjun Gao, Wenkai Yang, Wenlve Huang, Wenqing Hou, Wentao Zhang, Wenting Ma, Xi Gao, Xiang He, Xiangwen Wang, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingchen Liu, Xingkai Yu, Xingyou Li, Xinyu Yang, Xinyu Zhang, Xu Chen, Xuanyu Wang, Xuecheng Su, Xueyin Chen, Xuheng Lin, Xuwei Fu, YC Yan, YQ Wang, YW Ma, Yanfeng Luo, Yang Zhang, Yanhong Xu, Yanru Ma, Yanwen Huang, Yao Li, Yao Li, Yao Xu, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Qian, Yi Shao, Yi Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yijia Wu, Yiliang Xiong, Yiling Ma, Ying He, Ying Tang, Ying Zhou, Yingjia

Luo, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiang Zhang, Yixiao Chen, Yixuan Tan, Yixuan Wei, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo, Yongtong Wu, Yu Wu, YuKun Li, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yuanhao Li, Yuduan Wang, Yuehan Yang, Yuer Xu, Yuhan Wu, Yuhao Meng, Yuheng Zou, Yukun Zha, Yunfan Xiong, Yupeng Chen, Yuping Lin, Yuqian Cao, Yuqian Wang, Yushun Zhang, Yuting Yan, Yutong Lin, Yuxian Gu, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuxuan Zhou, Yuyang Zhou, Yuzhen Huang, ZF Wu, Zehao Wang, Zehua Zhao, Zehui Ren, Zekai Zhang, Zhangli Sha, Zhe Fu, Zhe Ju, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zheren Gao, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhixuan Chen, Zhiyu Wu, Zhizhou Ren, Zhongyu Wu, Zhuoshu Li, Zhuping Zhang, Zian Xu, Zihao Wang, Zihua Qu, Zihui Gu, Zijia Zhu, Zilin Li, Zipeng Zhang, Ziwei Xie, Ziyi Gao, Ziyi Wan, Zizheng Pan, and Zongqing Yao. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026. https://arxiv.org/abs/2606.19348.

Guanting Dong, Junting Lu, Junjie Huang, Wanjun Zhong, Longxiang Liu, Shijue Huang, Zhenyu Li, Yang Zhao, Xiaoshuai Song, Xiaoxi Li, Jiajie Jin, Yutao Zhu, Hanbin Wang, Fangyu Lei, Qinyu Luo, Mingyang Chen, Zehui Chen, Jiazhan Feng, Ji-Rong Wen, and Zhicheng Dou. Agent-world: Scaling real-world environment synthesis for evolving general agent intelligence. *CoRR*, abs/2604.18292, 2026a. doi: 10.48550/ARXIV.2604.18292. https://doi.org/10.48550/arXiv.2604.18292.

Guanting Dong, Xinyu Yang, Jiajie Jin, Yuyang Hu, Xiaoshuai Song, Yifei Chen, Xiaoxi Li, Jiejun Tan, Yutao Zhu, Tat-Seng Chua, Ji-Rong Wen, and Zhicheng Dou. Agent-reflex: Teaching language agents to act reflectively, 2026b. https://arxiv.org/abs/2610.17953.

Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 226–231, 1996.

Runnan Fang, Shihao Cai, Baixuan Li, Jialong Wu, Guangyu Li, Wenbiao Yin, Xinyu Wang, Xiaobin Wang, Liangcai Su, Zhen Zhang, Shibin Wu, Zhengwei Tao, Yong Jiang, Pengjun Xie, Fei Huang, and Jingren Zhou. Towards general agentic intelligence via environment scaling, 2025. https://arxiv.org/abs/2509.13311.

Jiacheng Guo, Ling Yang, Peter Chen, Qixin Xiao, Yinjie Wang, Xinzhe Juan, Jiahao Qiu, Ke Shen, and Mengdi Wang. Genenv: Difficulty-aligned co-evolution between llm agents and environment simulators, 2025. https://arxiv.org/abs/2512.19682.

Harvey AI. Harvey lab: The legal agent benchmark, 2026. https://github.com/harveyai/harvey-labs/tree/v1.0. Announcement: https://www.harvey.ai/blog/introducing-harveys-legal-agent-benchmark.

Hang He, Linting Wang, Guanming Liu, Bo Mao, Jinxiu Liu, Yiran Suo, Maosen Zhou, Zhiheng Xi, Guanting Dong, Zhou Shao, Rui Zheng, Jie Zhou, Chengcheng Wan, Tao Gui, Qi Zhang, Xipeng Qiu, and Xuanjing Huang. Workrover: Scaling long-horizon cowork agent training via recursive environment evolution, 2026. https://arxiv.org/abs/2610.18067.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-bank: A comprehensive benchmark for tool-augmented LLMs. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3102–3116, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.187. https://aclanthology.org/2023.emnlp-main.187/.

Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli

Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianfu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han-Chung Lee. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *CoRR*, abs/2602.12670, 2026a. https://arxiv.org/abs/2602.12670.

Yuetai Li, Huseyin A Inan, Xiang Yue, Wei-Ning Chen, Lukas Wutschitz, Janardhan Kulkarni, Radha Poovendran, Robert Sim, and Saravan Rajmohan. Simulating environments with reasoning models for agent training, 2025. https://arxiv.org/abs/2511.01824.

Yuetai Li, Yichen Feng, Zhangchen Xu, Zixian Ma, Kaiyuan Zheng, Fengqing Jiang, Xinghua Sun, Rulin Shao, Zichen Chen, Yue Huang, Xinyang Han, Brian Lee, Kayla Xu, Shenglai Zeng, Hang Hua, Xiangliang Zhang, Basel Alomair, Ranjay Krishna, Luke Zettlemoyer, Pang Wei Koh, Bhaskar Ramasubramanian, Luyao Niu, Xiang Yue, and Radha Poovendran. Jobbench: Aligning agent work with human will, 2026b. https://arxiv.org/abs/2605.26329.

Bo Mao, Hang He, Linting Wang, Lizhi Lin, Maosen Zhou, Guanming Liu, Jinxiu Liu, Tianyu Huai, Chaoyun Zhang, Bingxuan Li, Kepeng Lei, Guanting Dong, Zhou Shao, Rui Zheng, Hang Yan, Jie Zhou, Chengcheng Wan, Tao Gui, Liang He, and Xipeng Qiu. Weft: Scaling tool-use post-training for general-purpose agents, 2026. https://arxiv.org/abs/2609.36887.

Meta. Muse Glimmer 30B: An on-device agentic model. https://huggingface.co/meta-models/Muse-Glimmer-30B, 2026. Official model card; accessed 26 Oct. 2026.

Moonshot AI. Kimi K3. https://huggingface.co/moonshotai/Kimi-K3, 2026. Official model card and technical report; accessed 26 Oct. 2026.

Nex-AGI. Nex-N2-mini. https://huggingface.co/nex-agi/Nex-N2-mini, 2026. Official model card; accessed 26 Oct. 2026.

OpenAI. GPT-5.6: Frontier intelligence that scales with your ambition. https://openai.com/index/gpt-5-6/, 2026a. Official announcement of the GPT-5.6 family incl. GPT-5.6 Sol, 9 Jul. 2026; accessed 26 Oct. 2026.

OpenAI. Addendum: Gpt-6.1 sol system card. https://cdn.openai.com/pdf/38e3efcf-545e-44cd-99ec-2b7eb395f4cc/oai_GPT_6_1_Sol.pdf, September 2026b. Released September 29, 2026.

Krista Opsahl-Ong, Arnav Singhvi, Jasmine Collins, Ivan Zhou, Cindy Wang, Ashutosh Baheti, Owen Oertell, Jacob Portes, Sam Havens, Erich Elsen, Michael Bendersky, Matei Zaharia, and Xing Chen. Officeqa pro: An enterprise benchmark for end-to-end grounded reasoning. *CoRR*, abs/2603.08655, 2026. https://arxiv.org/abs/2603.08655.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis, 2023. https://arxiv.org/abs/2305.15334.

Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR, 2025. https://proceedings.mlr.press/v267/patil25a.html.

Akshara Prabhakar, Zuxin Liu, Ming Zhu, Jianguo Zhang, Tulika Awalgaonkar, Shiyu Wang, Zhiwei Liu, Haolin Chen, Thai Hoang, Juan Carlos Niebles, Shelby Heinecke, Weiran Yao, Huan Wang, Silvio Savarese, and Caiming Xiong. Apigen-mt: Agentic pipeline for multi-turn data generation via simulated agent-human interplay, 2025. https://arxiv.org/abs/2504.03601.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023. https://arxiv.org/abs/2307.16789.

Qwen Team. Qwen3.6-27B: Flagship-level coding in a 27B dense model, April 2026a. https://qwen.ai/blog?id=qwen3.6-27b.

Qwen Team. Qwen3.6-35B-A3B. https://huggingface.co/Qwen/Qwen3.6-35B-A3B, 2026b. Official model card; accessed 26 Oct. 2026.

Qwen Team. Qwen3.8-Max: A new bar for coding and cowork. https://qwen.ai/blog?id=qwen3.8, 2026c. Official release post; accessed 26 Oct. 2026.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 68539–68551. Curran Associates, Inc., 2023. doi: 10.52202/075280-2997. https://proceedings.neurips.cc/paper_files/paper/2023/file/d842425e4bf79ba039352da0f658a906-Paper-Conference.pdf.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. https://arxiv.org/abs/2402.03300.

Daniel Shepard and Robin Salimans. Automationbench, 2026. https://arxiv.org/abs/2604.18934.

Quan Shi, Alexandra Zytek, Pedram Razavi, Karthik R Narasimhan, and Victor Barres. τ-knowledge: Evaluating conversational agents over unstructured knowledge. In Tong Zhang, Miroslav Dudik, Martin Jaggi, Alekh Agarwal, Sharon Li, Dale Schuurmans, Jerry Zhu, Felix Berkenkamp, Hanze Dong, and Alberto Bietti, editors, *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 112443–112471. PMLR, 06–11 Jul 2026. https://proceedings.mlr.press/v306/shi26ah.html.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652. Curran Associates, Inc., 2023. doi: 10.52202/075280-0377. https://proceedings.neurips.cc/paper_files/paper/2023/file/1b44b878bb782e6954cd888628510e90-Paper-Conference.pdf.

Xiaoshuai Song, Haofei Chang, Guanting Dong, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. Envscaler: Scaling tool-interactive environments for llm agent via programmatic synthesis, 2026. https://arxiv.org/abs/2601.05808.

Yifan Song, Weimin Xiong, Dawei Zhu, Wenhao Wu, Han Qian, Mingbo Song, Hailiang Huang, Cheng Li, Ke Wang, Rong Yao, Ye Tian, and Sujian Li. Restgpt: Connecting large language models with real-world restful apis, 2023. https://arxiv.org/abs/2306.06624.

Zhengwei Tao, Jialong Wu, Wenbiao Yin, Junkai Zhang, Baixuan Li, Haiyang Shen, Kuan Li, Liwen Zhang, Xinyu Wang, Yong Jiang, Pengjun Xie, Fei Huang, and Jingren Zhou. Webshaper: Agentically data synthesizing via information-seeking formalization, 2025. https://arxiv.org/abs/2507.15061.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for ai software developers as generalist agents, 2025. https://arxiv.org/abs/2407.16741.

Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent world model: Infinity synthetic environments for agentic reinforcement learning, 2026. https://arxiv.org/abs/2602.10090.

Jialong Wu, Baixuan Li, Runnan Fang, Wenbiao Yin, Liwen Zhang, Zhengwei Tao, Dingchu Zhang, Zekun Xi, Gang Fu, Yong Jiang, Pengjun Xie, Fei Huang, and Jingren Zhou. Webdancer: Towards autonomous information seeking agency, 2025. https://arxiv.org/abs/2505.22648.

Zhangchen Xu, Adriana Meza Soria, Shawn Tan, Anurag Roy, Ashish Sunil Agrawal, Radha Poovendran, and Rameswar Panda. Toucan: Synthesizing 1.5m tool-agentic data from real-world mcp environments, 2025. https://arxiv.org/abs/2510.01179.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. https://arxiv.org/abs/2505.09388.

John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 50528–50652. Curran Associates, Inc., 2024. doi: 10.52202/079017-1601. https://proceedings.neurips.cc/paper_files/paper/2024/file/5a7c947568c1b1328ccc5230172e1e7c-Paper-Conference.pdf.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. https://arxiv.org/abs/2210.03629.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024. https://arxiv.org/abs/2406.12045.

Fan Yin, Zifeng Wang, I-Hung Hsu, Jun Yan, Ke Jiang, Yanfei Chen, Jindong Gu, Long Le, Kai-Wei Chang, Chen-Yu Lee, Hamid Palangi, and Tomas Pfister. Magnet: Multi-turn tool-use data synthesis and distillation via graph translation. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32600–32616, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1566. https://aclanthology.org/2025.acl-long.1566/.

Bofei Zhang, Zirui Shang, Zhi Gao, Wang Zhang, Rui Xie, Xiaojian Ma, Tao Yuan, Xinxiao Wu, Song-Chun Zhu, and Qing Li. Tongui: Internet-scale trajectories from multimodal web tutorials for generalized gui agents, 2025. https://arxiv.org/abs/2504.12679.

Zhipu AI. GLM-5.2: Built for long-horizon tasks. https://huggingface.co/zai-org/GLM-5.2, 2026a. Official model card; accessed 26 Oct. 2026.

Zhipu AI. GLM-5.3: Frontier coding with emergent cyber capabilities. https://z.ai/blog/glm-5.3, 2026b. Official release post, 14 Aug. 2026; accessed 26 Oct. 2026.